Løse reelle problemer

Løse problemer med data fra fil, samt parameteroverføring og funksjoner

IN1000, uke 5
Geir Kjetil Sandve

Tilbakeblikk

- Dere bør nå beherske det sentrale fra uke 1 og 2:
  - Uttrykk, typer, variabler, beslutninger (if)
- Uke 3-4 introduserte vi to helt fundamentale konsept
  - **løkker** for å kjøre kode mange ganger (while/for)
  - **samlinger** for å holde på mange verdier (liste/mengde/ordbok)
- Hvor mye må dere ha skjønt av samlinger og løkker?
  - Ikke forventet stålkontroll i dag - krever modning
  - Det løsner etterhvert - trene, trene, trene
  - Mindre enn 13 timer med faget er på eget ansvar

Mål for uken

- Få enda mer trening i hvordan bruke løkker, samlinger og beslutninger for å løse problemer
  - Som del av dette skal vi lære å behandle data fra filer
- Lære om parameteroverføring og returverdier
  - Gjør at vi kan gjenbruke kode,
    samt lage større programmer uten å miste oversikt

Outline

- Lese, bruke og skrive data i filer
- Eksempel: kombinere data fra ulike filer
- Prosedyrer med parametre
- Funksjoner (returverdier)

Outline

- **Lese, bruke og skrive data i filer**
- Eksempel: kombinere data fra ulike filer
- Prosedyrer med parametre
- Funksjoner (returverdier)

Innlesing fra fil

- Å hente data fra filer er gøy!
  - Man kan jobbe på mye større og mer spennende data enn fra tastatur
  - Man slipper å taste det inn hver gang man kjører
- Innlesing fra tekstfiler er veldig rett frem i Python
  - En tekstfil åpnet i Python er faktisk en *samling* av linjer
  - Man kan dermed iterere gjennom linjer i filen vha for-løkke
  - (Det finnes også mange andre måter å gjøre det på)

Hvordan lese inn fra fil

- Først åpne en fil *(biblioteksfunksjonen **open** lager et fil-objekt)*:
  - **min_fil = open("mittFilNavn.txt")**
- Deretter iterere gjennom hver linje i filen:
  - min_fil = open("mittFilNavn.txt")
    for linje in min_fil:
- Inni for-løkken kan man gjøre noe med linjen
  - min_fil = open("mittFilNavn.txt")
    for linje in min_fil:
      print("Her fant jeg: " + linje)
- [les_fra_fil.py]

Organisere informasjon innad på linjer

- Det er ofte nyttig å organisere informasjon på bestemte måter i en fil:
  - Kan gruppere: én gruppe på hver linje, og alt tilhørende gruppen inndelt langs linjen
  - Representere en tabell: hver linje er en rad, og hver linje er videre inndelt i verdier per kolonne (som i et regneark)
- Slike filer er ofte i *tabulært format,* hvor man bruker et spesifikt tegn (f.eks. komma) for å skille verdier innad i linje

Lese tabulære filer i python

- En tabulær fil er en helt vanlig tekstfil, hvor man kan iterere gjennom linjer
  - for linje in open("min_tabular_fil.csv"):
- Hver linje kan splittes til en liste *(basert på separator-tegn)*
  - biter = linje.split(",") *#Splitter på hvert komma*
  - biter = linje.split() *#Splitter på blanke tegn*
- Man kan hente ut bestemte biter (kolonner):
  - navn = biter[0]
    alder = int(biter[1])
- Man kan også iterere gjennom bitene:
  - for bit in biter:
        print("Jeg fant: " + bit)

Bygge samling fra innhold i fil

- Vi har tidligere laget to versjoner av hovedstad-program:
  - I uke 1 sjekket vi land med if-setninger og printet hovedstad
  - I uke 3 la vi land og hovedsted inn i en ordbok og slo opp land
- Nå vil vi bygge ordboken fra innholdet i en fil
  - Vi trenger informasjon som kobler étt land med én hovedstad
  - Bruker tabulær fil hvor hver linje har land og tilhørende hovedstad
  - Går gjennom hver linje i fil, splitter, og legger inn i ordbok
- [hovedstad.py]

Oppsummere data gruppert på linjer

- Vi vil finne hvilken måned hadde høyest dagsnedbør
  - Har tabulær fil med nebørsmengde per dag gjennom året
  - Dagsnedbør er gruppert på én linje for hver måned
- Fremgangsmåte: nøstet løkke inkludert splitting
  - Itererer gjennom måneder (linjer)
  - Splitter hver linje til liste over dagsnedbør
  - Itererer gjennom dagsnedbør og finner høyeste
  - Skriver ut høyeste verdi for denne måneden
- [regn.py]

Oppgave

- Gitt en tekstfil av typen vist under (navn og alder), skriv et program som leser filen og skriver ut navnet på den eldste personen i fila

alder.csv

odlaug:76
oluf:65
gunda:74
malfrid:80
godtfred:68

En mulig løsning

- [alder.py]

Hvordan skrive til tekstfil

- Først åpne en fil for skriving:
  - min_fil = open("mittUtFilNavn.txt", **"w"**)
- Deretter skrive tekst (en streng-verdi) til slutten av filen:
  - min_fil = open("mittUtFilNavn.txt", "w")
    min_fil.write("Dette havner i filen")
- Man vil typisk ha linjeskift i det man skriver ut:
  - **min_fil.write("Dette havner i filen\n")**
    min_fil.write("Dette havner på\n ulike\n linjer\n")
- Når man er ferdig bør man lukke filen:
  - min_fil.close()
- [elefanter.py]

Outline

- Lese, bruke og skrive data i filer
- **Eksempel: kombinere data fra ulike filer**
- Prosedyrer med parametre
- Funksjoner (returverdier)

Eksempel: drømmer vi oss bort når det regner?

- Problemstilling:
  - Er det slik at nordmenn gjør flere internettsøk etter restplass på dager hvor det regner?
- Fremgangsmåte:
  - Lese nedbørsdata fra fil (hentet fra yr.no)
  - Lese søkestatistikk fra fil (hentet fra Google Trends)
  - Se om antall søk på "restplass" er høyere på dagene det var regn (mer enn 2mm)

Koble data

- Vi vil telle antall søk på dager det er regn
  - Vi kan skrive noe a la:
    if regn_denne_dagen:
        sum_sok_regndager += sok_denne_dagen
- Vi må da koble nedbør og søkeantall for samme dato
  (vi kan iterere gjennom datoer, men må få til at vi for en gitt dato samtidig får tak i *regn_denne_dagen* og *sok_denne_dagen)*
  - Informasjon om nedbør og søk for en bestemt dag ligger imidlertid i to separate filer
- Hvordan få tak i regn og søk for en gitt dag samtidig?
  - *(f.eks. skrive ut regn og søk på én linje for hver dag)*
  - Er det noe fra tidligere ukers pensum som kan hjelpe?
    Tenk først selv, deretter diskuter med sideperson!

Koble data

- En ordbok lar oss enkelt gå fra en dato til en verdi
- Vi kan gå gjennom nedbørsfilen og bygge en ordbok som kobler dato til nedbør (og gjøre samme for søk)
- Etterpå itererer vi gjennom datoer
  - Når vi behandler en gitt dato slår vi opp korresponderende søk og nedbør i ordbøkene
- [restplass1.py]

Gjøre selve analysen

- Når vi har fått tak i søk og nedbør for samme dag
  - I stedet for å skrive ut finner vi gjennomsnittlig søkescore for gode dager og for dårlige dager
  - Vi teller antall og summerer søkescore for gode dager (og gjør det samme for dårlige)
  - Til slutt regner vi ut gjennomsnitt (sum/antall)
- [restplass2.py]

Outline

- Lese, bruke og skrive data i filer
- Eksempel: kombinere data fra ulike filer
- **Prosedyrer med parametre**
- Funksjoner (returverdier)

Flere typer subrutiner

- **Subrutine** (fra uke 2): en **navngitt blokk** med kodelinjer, som kan **kalles** og **tilpasses**
- Vi vil i dag introdusere flere aspekter
  - Uke 2: Prosedyre - **uten** parametre og returverdi
  - I dag: Prosedyre **-** med **parametre**
  - I dag: Funksjon **-** med **returverdi**
  - Om tre uker: Instans**-metode** (OO)

Prosedyrer **med** parametre

- Prosedyren vi så på i tidligere uke gjorde alltid eksakt det samme når den ble kallet
  - Det er sjelden av nytte!
- For å være nyttig må en slik prosedyre kunne **tilpasses**
  - Det gjør vi ved å sende inn **parametre**

Din første prosedyre med parametre

- **print** er en prosedyre hvor utfallet tilpasses†
- print(text):
  skriver verdien i text til skjermen
  - Variabelen text er en **parameter**
  - Verdien vi gir inn (f.eks. "hallo IN1000") når vi kaller print er et **argument**
- Parameter og argument er to sider av det samme
  - Parameter: **variabel** i prosedyre som tar i mot verdi
  - Argument: **verdi** sendt inn når prosedyren kalles

Prosedyre med parametre

def **mittProsedyreNavn**(parameter1, parameter2, ...):
  kodelinje1
  kodelinje2
  ...

For å kjøre alle kodelinjene i prosedyren ("*kalle* prosedyren"):

mittProsedyreNavn(argument1, argument2, ...)

Prosedyre med parametre

- {prosedyre_med_parameter_v1.py-prosedyre_med_parameter_v3.py}

Parameteren tilordnes verdien av argumentet!

```
def skrivAlder(alder):
    =
    if alder > 6:
        print("Velkommen til mitt program");
    else:
        print("Gaa heller ut og lek i skogen");

innlest = int(input("Hvor gammel er du? "))
skrivAlder(innlest)
```

Parameteren tilordnes verdien av argumentet!

```
def skrivAlder(alder):
    if alder > 6:
        print("Velkommen til mitt program");
    else:
        print("Gaa heller ut og lek i skogen");

print("Hacket av en toaaring: ")
skrivAlder(2)
```

Parameteren tilordnes verdien av argumentet!

```
def skrivAlder(alder):
    alder =  2
    if alder > 6:
        print("Velkommen til mitt program");
    else:
        print("Gaa heller ut og lek i skogen");

print("Hacket av en toaaring: ")
skrivAlder(2)
```

Oppgave:
Hva skrives ut på skjermen?

```
def pros1(a):
    print(a*2)

def pros2(b):
    print(b)
    pros1(b+2)

pros1(5)
pros2(4)
```

Løsning:
Hva skrives ut på skjermen?

```
def pros1(a):

    print(a*2)

def pros2(b):
    print(b)
    pros1(b+2)

pros1(5)
pros2(4)
```

På skjermen:

Løsning:

Hva skrives ut på skjermen?

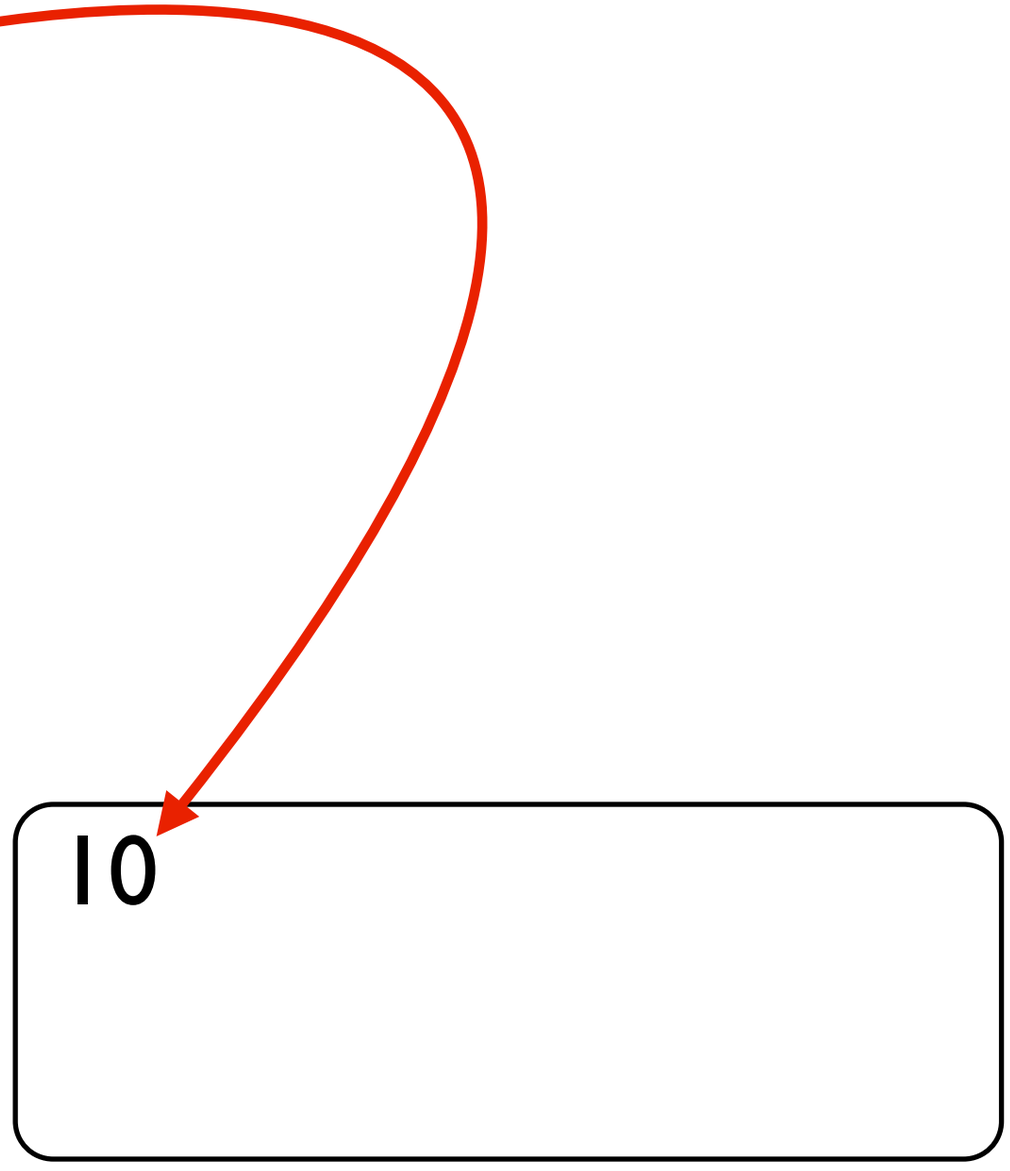
```
def pros1(a):
    a=5
    print(a*2) #10

def pros2(b):

    print(b)
    pros1(b+2)

pros1(5)
pros2(4)
```

På skjermen:



Løsning:
Hva skrives ut på skjermen?

```
def pros1(a):

    print(a*2)

def pros2(b):
    b=4
    print(b)
    pros1(b+2) #6

pros1(5)
pros2(4)
```



Løsning:

Hva skrives ut på skjermen?

```
def pros1(a):
    a=6
    print(a*2) #12

def pros2(b):
    b=4
    print(b)
    pros1(b+2) #6

pros1(5)
pros2(4)
```

På skjermen:

```
10
4
12
```

Prosedyrer outsourcer detaljer og håndterer redundans

- Man har ofte behov for (omtrent) samme funksjonalitet ulike steder i en kode
- Man ønsker da ikke å duplisere koden
  - Minsker oversiktlighet av kode
  - Endringer og rettinger må utføres mange steder
- Man samler i stedet funksjonaliteten i en prosedyren og kaller metoden der den trengs
  - Dersom det er noe variasjon i hva man trenger, representerer man det som varierer med en parameter

Outline

- Lese, bruke og skrive data i filer
- Eksempel: kombinere data fra ulike filer
- Prosedyrer med parametre
- **Funksjoner (returverdier)**

Subrutiner med returverdi:
Funksjoner

```
def mitt_funksjons_navn(parameter1, ...):
  kodelinje1
  kodelinje2
  return beregnet_verdi
```

For å kjøre alle kodelinjene i funksjonen ("*kalle* funksjonen"):

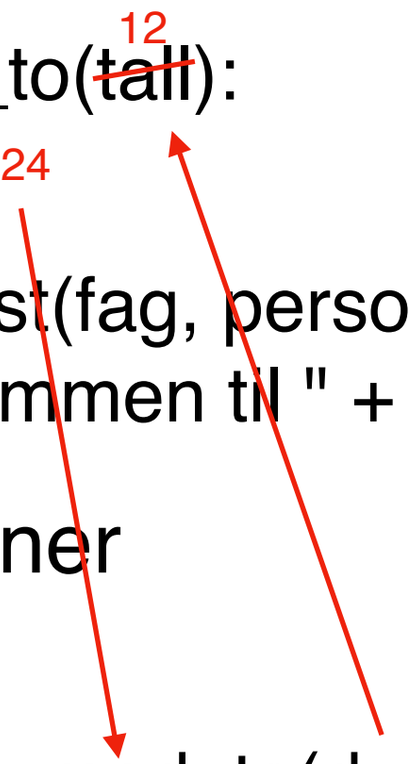
```
verdien_jeg_trenger = mitt_funksjons_navn(argument1, ..)
```

Subrutiner med returverdi: **Funksjoner**

- En funksjon lar deg outsource en beregning til en separat kodeblokk
  - Funksjonen tar inn en eller flere verdier,
    gjør en bestemt beregning,
    og sender tilbake resultatet
- Det som skiller en funksjon fra en prosedyre er altså at en funksjon returnerer noe:
  - def min_funksjon(parametre, ..):
    ...
    return en_beregnet_verdi

Subrutiner med returverdi: **Funksjoner**

- Eksempler på funksjoner:
  - def gang_med_to(~~tall~~):
    return ~~tall*2~~



  - def lag_velkomst(fag, person):
    return "Velkommen til " + fag + " kjaere " + person

- Bruk av funksjoner

```
dusin=12
to_dusin = gang_med_to(dusin)
print(to_dusin) #Skriver ut 24

velkomst = lag_velkomst("in1000", "Geir")
print(velkomst) #Velkommen til in1000 kjaere Geir
```

Prøv selv

(Lett modifisert fra eksamen 2014)

Skriv en funksjon

def pris(gratis, alder):

Dersom parameteren **gratis** har verdien **True**, skal funksjonen *alltid* returnere 0. Dersom parameteren **gratis** har verdien **False** og verdien av **alder** er mindre enn 18, skal funksjonen returnere 100, ellers 200. Altså skal f.eks. kallet **pris (true, 10)** returnere 0, kallet **pris(false,10)** returnere 100 og kallet **pris(false, 50)** returnere 200.

En mulig løsning

```
def pris(gratis, alder):
    if gratis:
        svar = 0
elif alder < 18:
    svar = 100
else:
    svar = 200

return svar
```

En annen mulig løsning

*(returnere inni **if**)*

```
def pris(gratis, alder):
    if gratis:
        return 0
    elif alder < 18:
        return 100
    else:
        return 200
```

En tredje mulig løsning

*(bare rene **if** - ingen else..)*

```
def pris(gratis, alder):
    if gratis:
        svar = 0
if (not gratis and alder < 18):
    svar = 100
if (not gratis and alder >= 18):
    svar = 200

return svar
```

Introdusere funksjoner i programmet om restplass

- Å lage nedbørs-ordbok fra data i fil er én oppgave
  - Vi definerer en funksjon som basert på et filnavn lager og returnerer en ordbok [restplass3.py]
- Den resterende analysen kan legges i en prosedyre
  - Basert på ordbok for regn og filnavn for søkeantall skriver den ut gjennomsnittlig søk på gode og dårlige dager
- Kallene til funksjonen og prosedyren over legges i en funksjon "hovedprogram"
  - Tydeliggjør hva som kjøres og sikrer at alle verdier overføres med parametre og returverdier [restplass4.py]

Oppsummering

- Å hente data fra filer er gøy!
  - Kan jobbe med store og reelle data, uten tasting
- Prosedyrer og funksjoner gjør livet behagelig!
  - Tillater å tenke overordnet først,
    og deretter ordne detaljene
- Når alt dere har lært frem til nå kobles sammen,
  kan det brukes til å utrette mye